

A Hybrid Architecture for 2D Game Development: The Case of Asteroid Miner

Luis Alejandro Morales
Systems Engineering Student
Universidad Distrital Francisco José de Caldas
lamoralesn@udistrital.edu.co

Juan Manuel Otálora Hernández
Systems Engineering Student
Universidad Distrital Francisco José de Caldas
jmotalorah@udistrital.edu.co

Julian Felipe Donoso
Systems Engineering Student
Universidad Distrital Francisco José de Caldas
jfdonosoh@udistrital.edu.co

Abstract—The development of optimization-based video games requires the integration of efficient algorithms, data structures, and software architectures capable of supporting real-time interaction. This paper presents Asteroid Miner, a hybrid Python–C++ game in which players collect resources from asteroids under fuel constraints. The proposed solution combines a greedy algorithm for real-time recommendations, a backtracking algorithm with pruning to compute optimal routes, a linked list for route history management, and a binary search tree for asteroid catalog organization. Experimental results demonstrate successful integration between Python and C++ through JSON-based communication, while providing optimal route evaluation, persistent mission statistics, and efficient data management.

Index Terms—Project guidelines, IEEE LaTeX, engineering education, templates

I. INTRODUCTION

The design of intelligent systems for video games has become increasingly relevant as modern applications demand efficient decision-making under constrained environments. In particular, games involving resource management and navigation require algorithms capable of balancing optimality and computational efficiency. These challenges are commonly addressed using classical algorithmic paradigms such as greedy methods and backtracking techniques.

Greedy algorithms are widely used in scenarios where locally optimal decisions can lead to satisfactory global solutions, especially in real-time environments due to their low computational cost. On the other hand, backtracking provides a systematic way to explore multiple possible solutions, making it suitable for problems with constraints such as limited resources. Additionally, efficient data structures such as lists and binary search trees (BST) play a crucial role in organizing and retrieving information within dynamic systems.

Resource-constrained path optimization problems appear in many domains, including logistics, robotics, autonomous navigation, and video games. These problems are closely related to variants of the Traveling Salesman Problem (TSP) and Knapsack optimization, where a limited resource budget restricts the feasible search space. Classical approaches include greedy heuristics for fast approximations and exhaustive search techniques such as backtracking for optimal solutions.

This paper presents the design and implementation of *Asteroid Miner*, a 2D game developed using Python and C++, where different algorithmic strategies are integrated into a unified architecture. The system uses a greedy approach to prioritize asteroid mining based on value-distance ratio, while a backtracking algorithm determines optimal routes under fuel constraints. Furthermore, C++ is used to manage route history through lists and to implement an asteroid catalog using a binary search tree.

The main contribution of this work is the integration of multiple computational techniques within an interactive environment, demonstrating how algorithmic theory can be applied to solve practical optimization problems in game development.

II. METHODS AND MATERIALS

A. Problem Formulation

The problem addressed in this work can be formulated as a constrained optimization problem. Given a set of asteroids $A = \{a_1, a_2, \dots, a_n\}$, where each asteroid a_i has an associated value v_i and a position (x_i, y_i) in a two-dimensional space, the objective is to determine a sequence of asteroids to visit such that the total collected value is maximized.

The mining unit starts from a fixed base and has a limited fuel capacity. The fuel consumption is proportional to the Euclidean distance traveled between asteroids. Therefore, any feasible solution must satisfy the constraint that the total travel distance does not exceed the available fuel and that the unit is able to return safely to the base.

B. Game Model and Logic

The system is implemented as a 2D game consisting of multiple levels with increasing difficulty. Each level defines a different spatial distribution of asteroids and a fixed fuel budget. The player must select a sequence of asteroids to mine, aiming to maximize resource collection while ensuring a safe return to the base.

Mining an asteroid consumes both time and fuel, reinforcing the need for efficient route planning. At the end of each level, the player's performance is evaluated based on the total

resources collected and whether the player successfully returns to the base.

A scoring system based on efficiency is implemented using the optimal solution as a reference. Players achieving results equivalent to the optimal solution receive three stars. Those achieving at least 75% of the optimal value receive two stars, while those meeting a minimum threshold receive one star. Failure to return to the base or to meet the minimum resource requirement results in zero stars.

C. System Architecture

The software architecture is divided into two independent layers. The Python layer is responsible for game visualization, user interaction, level management, and algorithm execution. The C++ layer manages persistent data structures and mission records.

Communication between both layers is performed exclusively through JSON file exchange, satisfying the project constraints. When a mission ends, Python generates a *game_state.json* file containing the collected resources, achieved stars, visited route, and current level. The C++ engine reads this file, reconstructs the route history using a linked list, organizes mined asteroids within a binary search tree, and generates output files used by the statistics module.

This architecture ensures modularity, facilitates testing, and demonstrates interoperability between two programming languages without direct bindings or shared memory.

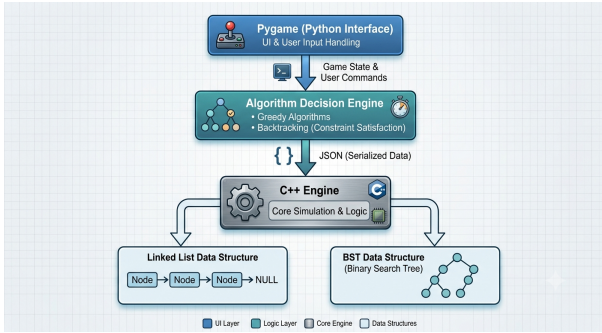


Fig. 1. Hybrid architecture of Asteroid Miner.

D. Complexity Analysis

The greedy recommendation algorithm evaluates every available asteroid once, resulting in a time complexity of $O(n)$. This makes it suitable for real-time assistance during gameplay.

The backtracking algorithm explores combinations of asteroid visits while respecting the fuel constraint. In the worst case, its complexity is $O(2^n)$. However, pruning techniques are applied whenever the remaining fuel is insufficient to continue exploration, significantly reducing the practical search space.

The linked list used for route history supports insertion in $O(n)$ time and traversal in $O(n)$. The binary search tree supports insertion and search operations in $O(\log n)$ on average and $O(n)$ in the worst case when the tree becomes unbalanced.

E. Algorithmic Approach

Algorithm 1 Game Execution Flow

```

1: Input: Set of asteroids  $A$ , fuel  $F$ 
2: Output: Player score
3: Initialize player at base
4: while  $fuel > 0$  and player selects asteroids do
5:   Move to selected asteroid
6:   Update fuel based on distance
7:   Mine asteroid and collect value
8: end while
9: Return to base
10: if player reaches base then
11:   Evaluate collected resources
12:   Assign stars based on performance
13: else
14:   Assign 0 stars
15: end if

```

Algorithm 2 Greedy Asteroid Selection

```

1: Input: Set of asteroids  $A$ , position  $p$ 
2: Output: Ordered list of asteroids
3: for each asteroid  $a_i \in A$  do
4:   Compute  $r_i = \frac{value_i}{distance(p, a_i)}$ 
5: end for
6: Sort asteroids in descending order of  $r_i$ 
7: Return sorted list

```

Algorithm 3 Optimal Route using Backtracking

```

1: Input: Set of asteroids  $A$ , fuel  $F$ 
2: Output: Best route
3:  $best\_value \leftarrow 0$ 
4:  $best\_route \leftarrow \emptyset$ 
5: Function Explore(route, remaining_fuel, value)
6: if  $r$  then
7:    $remaining\_fuel - cost - return\_cost \geq 0$ 
8:    $best\_value \leftarrow value$ 
9:    $best\_route \leftarrow route$ 
10: end if
11: for each asteroid  $a_i \notin route$  do
12:    $cost \leftarrow distance(current\_position, a_i)$ 
13:   if  $remaining\_fuel - cost \geq 0$  then
14:     Explore( $route \cup \{a_i\}$ , remaining_fuel - cost, value +  $v_i$ )
15:   end if
16: end for
17: Explore( $\emptyset$ ,  $F$ , 0)
18: return  $best\_route$ 

```

F. Data Structures and Implementation

C++ is used to implement efficient data structures required by the system. A list structure is used to store the history of visited asteroids, enabling efficient traversal and reconstruction

of routes. Additionally, a binary search tree (BST) is used to maintain an ordered catalog of asteroids based on their value, allowing efficient insertion, search, and retrieval operations [2].

The graphical interface is developed using Pygame, which provides tools for rendering the 2D environment, handling user input, and updating the game state in real time. This integration allows visualization of the decision-making process and facilitates validation of the implemented algorithms.

Although the proposed system is effective for static environments, it assumes that asteroid positions remain constant during gameplay, which may limit its applicability in dynamic scenarios.



Fig. 2. Prototype interface of Asteroid Miner showing asteroid distribution and player route.

III. RESULTS & DISCUSSION

Several experiments were conducted using the three game levels implemented in Asteroid Miner. The objective was to compare player-generated routes against the optimal routes produced by the backtracking algorithm.

The greedy algorithm consistently produced valid recommendations in real time and required negligible execution time. However, the suggested route was not always optimal because decisions were based solely on local value-to-distance ratios.

The backtracking algorithm successfully identified the optimal solution for every level tested. The implementation was enhanced using pruning strategies that discarded branches unable to satisfy fuel constraints, reducing the number of explored nodes and improving execution time.

Integration tests were performed to validate communication between Python and C++. At the end of each mission, Python generated a JSON file containing the route, resources, stars, and level information. The C++ engine correctly reconstructed the route history, stored asteroid information in the BST catalog, and generated mission summaries without errors.

User acceptance tests confirmed that players could complete missions, receive greedy recommendations, view route histories, and compare their results against the optimal solution generated by the backtracking algorithm.

IV. CONCLUSIONS

This project successfully demonstrated the integration of classical algorithmic techniques within an interactive game

environment. The combination of greedy heuristics and backtracking allowed both real-time decision support and optimal solution computation. The implementation of linked lists and binary search trees in C++ fulfilled the project requirements while providing persistent storage and efficient organization of game information.

The hybrid Python–C++ architecture proved effective for separating gameplay logic from data management. JSON-based communication enabled interoperability without requiring external libraries or language bindings.

Future work may include balancing the game using larger maps, implementing self-balancing trees such as AVL trees, incorporating dynamic asteroid generation, and exploring more advanced optimization techniques such as branch-and-bound or genetic algorithms.

ACKNOWLEDGMENTS

Optional: funding sources, collaborators who contributed to your work.

REFERENCES

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. MIT Press, 2009.
- [2] D. E. Knuth, *The Art of Computer Programming, Volume 3: Sorting and Searching*. Addison-Wesley, 1998.
- [3] Pygame Developers, *Pygame Documentation*. [Online]. Available: <https://www.pygame.org/docs/>
- [4] D. Crockford, *The application/json Media Type for JavaScript Object Notation (JSON)*. RFC 8259, 2017.
- [5] E. Horowitz, S. Sahni, and S. Rajasekaran, *Fundamentals of Computer Algorithms*. Universities Press, 2008.
- [6] M. Goodrich, R. Tamassia, and M. Goldwasser, *Data Structures and Algorithms in Python*. Wiley, 2013.